You:

Trong WPF C# của tội hiện tại có như sau Trong file .xaml: <UserControl x:Class="BookStoreManagement.Products.ProductsWindow" xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation" xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml" xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006" xmlns:d="http://schemas.microsoft.com/expression/blend/2008" xmlns:local="clr-namespace:BookStoreManagement" mc:Ignorable="d" Height="458" Width="690" Loaded="Product_Loaded"> <UserControl.Resources> <ResourceDictionary> <local:RelativeToAbsoluteConverter x:Key="AbsoluteConverter"/> <ResourceDictionary.MergedDictionaries> <ResourceDictionary Source="../Resource/UIResource.xaml"/> </ResourceDictionary.MergedDictionaries> </ResourceDictionary> </UserControl.Resources> <Grid> <Grid.RowDefinitions> <RowDefinition></RowDefinition> <RowDefinition Height="420"></RowDefinition> </Grid.RowDefinitions> <ListView Name="BookList" Grid.Row="1" Width="690" Height="420" Background="Cornsilk" ScrollViewer.HorizontalScrollBarVisibility="Disabled" HorizontalAlignment="Center" VerticalAlignment="Top"> <ListView.ItemContainerStyle > <Style TargetType="ListViewItem"> <Setter Property="Template"> <Setter.Value> <ControlTemplate TargetType="ListViewItem"> <Border Background="{TemplateBinding Background}" BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="{TemplateBinding BorderThickness}"> <ContentPresenter /> </Border> <ControlTemplate.Triggers> <Trigger Property="IsSelected" Value="True"> <Setter Property="BorderBrush" Value="Transparent" /> <Setter Property="Background" Value="Transparent" /> </Trigger> </ControlTemplate.Triggers> </ControlTemplate> </Setter.Value> </Setter> </Style> </ListView.ItemContainerStyle> <ListView.ItemTemplate> <DataTemplate> <Border Name="itemsBorder" CornerRadius="10" Background="AliceBlue" BorderBrush="Black" BorderThickness="2" Width="150" Height ="170" Margin="5, 5, 0, 0"> <Border.Effect> <DropShadowEffect Color="Black" Direction="320" ShadowDepth="5" BlurRadius="10" Opacity="0.5" /> </Border.Effect> <StackPanel VerticalAlignment="Center"> <Image Width="100" Height="100" Margin="0, 0, 0, 0" Source="{Binding ImageURL, Converter={StaticResource AbsoluteConverter}}"/> <TextBlock HorizontalAlignment="Center" Text="{Binding BookName}" FontSize="8" FontWeight="Medium" Margin="0, 10, 0, 10"> </TextBlock> <TextBlock HorizontalAlignment="Center" Text="{Binding Price}" FontSize="8" FontWeight="Medium"/> </StackPanel> </Border> </DataTemplate> </ListView.ItemTemplate> <ListView.ItemsPanel> <ItemsPanelTemplate> <WrapPanel/> </ItemsPanelTemplate> </ListView.ItemsPanel> </ListView> <!-- Pagination Controls --> <!--<Button Content="Previous" Click="Previous_Click" Width="100" Height="35" HorizontalAlignment="Left"/> <Button Content="Next" Click="Next_Click" Width="100" Height="35" HorizontalAlignment="Right"/>--> </Grid> </UserControl> Trong file .cs: using System; using System.Collections.Generic; using System.Linq; using System.Text; using System.Threading.Tasks; using System.Windows; using System.Windows.Controls; using System.Windows.Data; using System.Windows.Documents; using System.Windows.Input; using System.Windows.Media; using System.Windows.Media.Imaging; using System.Windows.Navigation; using System.Windows.Shapes; using System.Data; using System.ComponentModel; using BookStoreManagement.Entities; namespace BookStoreManagement.Products { /// <summary> /// Interaction logic for ProductsWindow.xaml /// </summary> public partial class ProductsWindow : UserControl { public ProductsWindow() { InitializeComponent(); } BindingList<Product> _lib = new BindingList<Product>(); private void Product_Loaded(object sender, RoutedEventArgs e) { string sql = @"SELECT * FROM Products"; DataTable dt = DBConnect.DB.LoadCSDL(sql); for (int i = 0; i < dt.Rows.Count; i++) { var book = new Product() { BookName = (string)dt.Rows[i]["ProductName"], ImageURL = (string)dt.Rows[i]["ImageURL"], Price = (decimal)dt.Rows[i]["Price"], }; _lib.Add(book); } BookList.ItemsSource = _lib; } } } Thực hiện các yêu cầu sau đây: 1. Viết tính năng phân trang cho chương trình 2. Viết tính năng tìm kiếm sách bằng cách nhập tên sách, bằng cách nhập kí tự đến đâu sẽ hiển thị gợi ý đến đó.

ChatGPT:

Để thêm các tính năng phân trang và tìm kiếm với gợi ý theo tên sách vào chương trình WPF của bạn, bạn sẽ cần thực hiện các bước sau:

1. Tính năng Phân trang

Để thêm tính năng phân trang, bạn cần một số thành phần UI để cho phép người dùng chuyển giữa các trang và một cách để quản lý số lượng mục hiển thị trên mỗi trang.

Bước 1: Thêm các nút điều hướng phân trang vào giao diện người dùng (UI) của bạn. Trong file .xaml, thêm hai nút `Previous` và `Next` như sau:

```
xml
<Button Content="Previous" Click="Previous_Click" Width="100" Height="35" HorizontalAlignment="Left"/>
<Button Content="Next" Click="Next_Click" Width="100" Height="35" HorizontalAlignment="Right"/>
```

Bước 2: Trong file code-behind (.cs), thêm các biến để quản lý phân trang:

```
csharp
private int currentPage = 1;
private int itemsPerPage = 10; // Thay đổi số lượng mục trên mỗi trang theo ý bạn
private int totalItems = 0;
```

Bước 3: Sửa đổi phương thức `Product_Loaded` để nó chỉ tải dữ liệu của trang hiện tại:

```
csharp
private void Product_Loaded(object sender, RoutedEventArgs e)
{
    LoadPage();
}

private void LoadPage()
{
    string sql = $"SELECT * FROM Products LIMIT {itemsPerPage} OFFSET {(currentPage - 1) * itemsPerPage}";
    // Thực hiện truy vấn và cập nhật _lib như trước
    // ...
    totalItems = // Tính tổng số mục
```

```
    BookList.ItemsSource = _lib;
}
```

Bước 4: Xử lý sự kiện click cho các nút `Previous` và `Next`:

```
csharp
private void Previous_Click(object sender, RoutedEventArgs e)
{
    if (currentPage > 1)
    {
        currentPage--;
        LoadPage();
    }
}

private void Next_Click(object sender, RoutedEventArgs e)
{
    if (currentPage * itemsPerPage < totalItems)
    {
        currentPage++;
        LoadPage();
    }
}
```

2. Tính năng Tìm kiếm sách với gợi ý

Tính năng tìm kiếm này yêu cầu một TextBox cho phép người dùng nhập vào và một cách để lọc dữ liệu dựa trên input.

Bước 1: Thêm một TextBox vào giao diện người dùng:

```
xml
<TextBox Width="200" Height="30" PlaceholderText="Search Book..." KeyUp="SearchTextBox_KeyUp"/>
```

Bước 2: Trong file .cs, xử lý sự kiện `KeyUp` để lọc dữ liệu:

```
csharp
private void SearchTextBox_KeyUp(object sender, KeyEventArgs e)
{
    var filtered = _lib.Where(book => book.BookName.ToLower().Contains((sender as TextBox).Text.ToLower())).ToList();
    BookList.ItemsSource = filtered;
}
```

Nhớ rằng bạn có thể cần tinh chỉnh cách lấy dữ liệu từ cơ sở dữ liệu hoặc cách lọc dữ liệu để phù hợp với yêu cầu cụ thể của ứng dụng của bạn. Đảm bảo kiểm tra và debug kỹ lưỡng sau khi thực hiện các thay đổi.

You:

private void LoadPage() { string sql = $"SELECT * FROM Products LIMIT {itemsPerPage} OFFSET {(currentPage - 1) * itemsPerPage}"; // Thực hiện truy vấn và cập nhật _lib như trước // ... totalItems = // Tính tổng số mục BookList.ItemsSource = _lib; } Ở đoạn code này: //Tính tổng số mục là sao

---

ChatGPT:

Trong đoạn code mà bạn đang xem xét, "// Tính tổng số mục" là một chú thích để chỉ ra rằng bạn cần tính tổng số mục (sản phẩm, sách, v.v.) có trong cơ sở dữ liệu. Điều này cần thiết để quản lý việc phân trang, như biết được tổng số trang và khi nào cần ẩn hoặc hiển thị các nút "Trước" và "Tiếp theo".

Để tính tổng số mục, bạn sẽ thực hiện một truy vấn SQL để đếm tổng số bản ghi trong bảng `Products`. Đây là một ví dụ về cách thực hiện điều này:

```
csharp
private void LoadPage()
{
    // Truy vấn để lấy dữ liệu cho trang hiện tại
    string sql = $"SELECT * FROM Products LIMIT {itemsPerPage} OFFSET {(currentPage - 1) * itemsPerPage}";
    // Thực hiện truy vấn và cập nhật _lib
    // ...

    // Truy vấn để lấy tổng số mục
    string countSql = "SELECT COUNT(*) FROM Products";
```

```
    totalItems = DBConnect.DB.ExecuteScalar(countSql); // Giả sử bạn có một phương thức ExecuteScalar để thực hiện truy vấn

    BookList.ItemsSource = _lib;
}
```

Trong đoạn code trên, `ExecuteScalar` là một phương thức giả định, bạn cần thay thế nó bằng phương thức thực tế mà bạn sử dụng để thực hiện truy vấn SQL và lấy kết quả. Hàm này sẽ trả về giá trị đếm tổng số hàng trong bảng `Products`, cho bạn biết tổng số mục bạn có.

You:

$"SELECT * FROM Products LIMIT {itemsPerPage} OFFSET {(currentPage - 1) * itemsPerPage}"; kIỂM Tra lại đoạn code SQL này

---

ChatGPT:

Đoạn SQL bạn đã viết sử dụng cú pháp phân trang với các từ khoá `LIMIT` và `OFFSET`, thường thấy trong các cơ sở dữ liệu SQL như PostgreSQL, MySQL, và SQLite. Tuy nhiên, nếu bạn đang sử dụng SQL Server, cú pháp sẽ khác một chút.

Đối với SQL Server, bạn sẽ sử dụng `OFFSET` và `FETCH NEXT` như sau:

```
sql
SELECT * FROM Products
ORDER BY [some_column] -- Thay thế [some_column] bằng tên cột mà bạn muốn sắp xếp
OFFSET {(currentPage - 1) * itemsPerPage} ROWS
FETCH NEXT {itemsPerPage} ROWS ONLY;
```

Ở đây, `ORDER BY` là bắt buộc khi sử dụng `OFFSET` và `FETCH` trong SQL Server. Bạn cần chọn một cột để sắp xếp kết quả truy vấn.

Đối với MySQL, PostgreSQL, và SQLite, cú pháp ban đầu của bạn là đúng:

```
sql
SELECT * FROM Products
LIMIT {itemsPerPage}
OFFSET {(currentPage - 1) * itemsPerPage};
```

Đảm bảo rằng bạn sử dụng cú pháp phù hợp với hệ quản trị cơ sở dữ liệu (DBMS) mà bạn đang sử dụng.

You:

SELECT * FROM Products ORDER BY [some_column] -- Thay thế [some_column] bằng tên cột mà bạn muốn sắp xếp OFFSET {(currentPage - 1) * itemsPerPage} ROWS FETCH NEXT {itemsPerPage} ROWS ONLY; Vậy biểu diễn đoạn code sql server này vào trong code c# cho tôi, cụ thể là vào trong đây string sql = @"...đoạn code sql đó..." ;

ChatGPT:

Để tích hợp đoạn truy vấn SQL Server với phân trang vào trong code C# của bạn, bạn cần chuyển đoạn truy vấn SQL vào một chuỗi string và sử dụng các biến `currentPage` và `itemsPerPage`. Đảm bảo rằng bạn đã xác định một cột để sắp xếp trong câu lệnh `ORDER BY`. Dưới đây là cách bạn có thể làm điều này:

```
csharp
private void LoadPage()
{
    // Chọn một cột để sắp xếp, ví dụ 'ProductID'
    string orderByColumn = "ProductID";

    // Tạo truy vấn SQL với phân trang
    string sql = $@"
        SELECT * FROM Products
        ORDER BY {orderByColumn}
        OFFSET {(currentPage - 1) * itemsPerPage} ROWS
        FETCH NEXT {itemsPerPage} ROWS ONLY";
```

```
    // Thực hiện truy vấn và cập nhật _lib như trước
    // ...

    // Tính tổng số mục
    string countSql = "SELECT COUNT(*) FROM Products";
    totalItems = DBConnect.DB.ExecuteScalar(countSql); // Giả sử bạn có một phương thức ExecuteScalar để thực hiện truy vấn

    BookList.ItemsSource = _lib;
}
```

Trong đoạn code trên, bạn cần thay thế `ProductID` bằng tên cột thực tế bạn muốn sử dụng để sắp xếp. `currentPage` và `itemsPerPage` là các biến mà bạn đã định nghĩa trước đó để quản lý trang hiện tại và số lượng mục trên mỗi trang.

Hãy chắc chắn rằng bạn đã xử lý các trường hợp ngoại lệ và kiểm tra kỹ càng để đảm bảo rằng truy vấn hoạt động chính xác với cơ sở dữ liệu của bạn.

You:

Vậy đoạn code của 2 nút chuyển trang qua lại sẽ như thế nào

ChatGPT:

Để xử lý sự kiện click cho các nút "Previous" và "Next" trong chức năng phân trang của bạn, bạn cần thêm hai phương thức vào file code-behind (.cs) của UserControl. Các phương thức này sẽ được kích hoạt khi người dùng nhấp vào các nút tương ứng và sẽ cập nhật trang hiện tại, sau đó tải lại dữ liệu cho trang đó. Dưới đây là cách bạn có thể làm điều đó:

```
csharp
private void Previous_Click(object sender, RoutedEventArgs e)
{
    if (currentPage > 1)
    {
        currentPage--;
        LoadPage();
    }
}

private void Next_Click(object sender, RoutedEventArgs e)
{
    // Ở đây bạn cần tính tổng số trang bằng cách chia tổng số mục cho số mục trên mỗi trang
    int totalPages = (int)Math.Ceiling((double)totalItems / itemsPerPage);

    if (currentPage < totalPages)
    {
        currentPage++;
        LoadPage();
    }
}
```

Trong các phương thức `Previous_Click` và `Next_Click`:

- `Previous_Click`: Kiểm tra xem trang hiện tại có lớn hơn 1 không (điều này có nghĩa là có trang trước đó để chuyển đến). Nếu có, giảm `currentPage` đi 1 và gọi `LoadPage()` để tải dữ liệu cho trang trước.
- `Next_Click`: Tính tổng số trang bằng cách chia tổng số mục cho số mục trên mỗi trang. Nếu `currentPage` nhỏ hơn tổng số trang, tăng `currentPage` lên 1 và gọi `LoadPage()` để tải dữ liệu cho trang tiếp theo.

Đảm bảo rằng bạn đã tính toán `totalItems` chính xác trong phương thức `LoadPage()` để xác định tổng số trang. Nếu bạn chưa tính `totalItems`, bạn cần thêm một truy vấn SQL vào `LoadPage()` để lấy tổng số mục từ cơ sở dữ liệu.

You:

Tôi thấy có vẻ 2 nút này bị sai

ChatGPT:

Để xử lý sự kiện click cho các nút "Previous" và "Next" trong chức năng phân trang của bạn, các phương thức `Previous_Click` và `Next_Click` cần được viết một cách cẩn thận để chúng xử lý đúng cách việc chuyển đổi giữa các trang. Dưới đây là cách bạn có thể cải tiến các phương thức này:

```
csharp
private void Previous_Click(object sender, RoutedEventArgs e)
{
    if (currentPage > 1)
    {
        currentPage--;
        LoadPage();
    }
}

private void Next_Click(object sender, RoutedEventArgs e)
{
    int totalPages = (int)Math.Ceiling((double)totalItems / itemsPerPage);
    if (currentPage < totalPages)
    {
        currentPage++;
        LoadPage();
    }
}
```

Trong các phương thức này:

- `Previous_Click`: Kiểm tra nếu `currentPage` lớn hơn 1, điều này có nghĩa là bạn không đang ở trang đầu tiên và có thể chuyển về trang trước. Giảm `currentPage` xuống 1 và gọi `LoadPage()` để tải dữ liệu cho trang hiện tại.
- `Next_Click`: Tính tổng số trang có thể có bằng cách chia `totalItems` cho `itemsPerPage` và làm tròn lên bằng `Math.Ceiling` để đảm bảo bạn có đủ trang cho tất cả các mục. Nếu `currentPage` nhỏ hơn `totalPages`, tăng `currentPage` lên 1 và gọi `LoadPage()` để tải dữ liệu cho trang kế tiếp.

Đảm bảo rằng `totalItems` đã được tính toán đúng cách trong phương thức `LoadPage()` hoặc một phương thức khởi tạo tương tự để bạn biết được tổng số mục bạn đang quản lý. Điều này quan trọng để xác định số lượng trang tổng cộng và khi nào nên kích hoạt các nút phân trang.

You:

Ở chức năng tìm kiếm sách trên của bạn { var filtered = _lib.Where(book => book.BookName.ToLower().Contains((sender as TextBox).Text.ToLower())).ToList(); BookList.ItemsSource = filtered; } Tôi muốn chức năng này xổ xuống sản phẩm ở thanh tìm kiếm cho người dùng thấy và chọn, còn ở màn hình chính vẫn giữ nguyên

---

ChatGPT:

Để thực hiện chức năng tìm kiếm với gợi ý xuất hiện dưới dạng một danh sách xổ xuống khi người dùng gõ vào thanh tìm kiếm, trong khi vẫn giữ nguyên danh sách sản phẩm ban đầu trên màn hình chính, bạn sẽ cần thêm một số điều chỉnh vào giao diện người dùng và logic xử lý phía sau.

Dưới đây là các bước cơ bản để thực hiện điều này:

1. Thêm ComboBox cho Gợi ý Tìm kiếm

Thay vì sử dụng một `TextBox` đơn giản, bạn có thể sử dụng một `ComboBox` trong WPF, vì nó hỗ trợ tính năng xổ xuống mà bạn cần.

Trong file .xaml của bạn, thêm một `ComboBox`:

```
xml
<ComboBox x:Name="SearchComboBox"
          Height="30"
          IsEditable="True"
          IsTextSearchEnabled="True"
          TextChanged="SearchComboBox_TextChanged">
</ComboBox>
```

Đảm bảo rằng `IsEditable` được đặt thành `True` để người dùng có thể nhập văn bản vào đó.

2. Xử lý Sự kiện Thay đổi Văn bản

Bây giờ, bạn cần xử lý sự kiện `TextChanged` cho `ComboBox`. Trong phương thức xử lý này, bạn sẽ lọc danh sách sản phẩm dựa trên văn bản nhập vào và cập nhật danh sách mục của `ComboBox`.

```
csharp
private void SearchComboBox_TextChanged(object sender, TextChangedEventArgs e)
{
    var searchText = (sender as ComboBox).Text.ToLower();
    var filteredItems = _lib.Where(book => book.BookName.ToLower().Contains(searchText)).ToList();

    SearchComboBox.ItemsSource = filteredItems;
    SearchComboBox.IsDropDownOpen = filteredItems.Any();
}
```

Trong đoạn code này, khi người dùng gõ vào `ComboBox`, danh sách `filteredItems` sẽ được cập nhật dựa trên từ khóa tìm kiếm, và `ComboBox` sẽ hiển thị danh sách này dưới dạng gợi ý xổ xuống.

3. Lựa Chọn Từ Danh sách Gợi ý

Bạn cũng cần xử lý sự kiện khi người dùng chọn một mục từ danh sách gợi ý. Bạn có thể thêm một sự kiện `SelectionChanged` vào `ComboBox` để xử lý điều này.

```
csharp
<ComboBox x:Name="SearchComboBox"
          ...
          SelectionChanged="SearchComboBox_SelectionChanged">
</ComboBox>
```

Và trong phương thức `SearchComboBox_SelectionChanged`, bạn có thể xử lý lựa chọn của người dùng:

```
csharp
private void SearchComboBox_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    // Xử lý lựa chọn của người dùng từ danh sách gợi ý
    // Ví dụ: có thể hiển thị thông tin chi tiết của sản phẩm được chọn
}
```

Nhớ rằng, khi sử dụng `ComboBox` như một thành phần xổ xuống, bạn cần đảm bảo rằng nó không làm ảnh hưởng đến trạng thái của danh sách sản phẩm chính trên màn hình. Thông thường, `ComboBox` sẽ chỉ được sử dụng để cung cấp gợi ý và có thể cần xử lý thêm để đáp ứng đúng yêu cầu của ứng dụng.

You:

Nó báo lỗi như vầy The property 'TextChanged' was not found in type 'ComboBox'.

ChatGPT:

Rất tiếc về sự nhầm lẫn trước đó. Trong WPF, `ComboBox` không có sự kiện `TextChanged` như `TextBox`. Để thực hiện tính năng tìm kiếm với gợi ý xuất hiện trong một danh sách xổ xuống, bạn sẽ cần sử dụng sự kiện `PreviewTextInput` hoặc kết hợp `TextBox` với `Popup` hoặc `ComboBox` để tạo ra giao diện người dùng tương tự.

Dưới đây là cách sử dụng `PreviewTextInput` để triển khai chức năng tìm kiếm với gợi ý:

Sử dụng `PreviewTextInput` trong `ComboBox`

1. **Cập nhật ComboBox trong XAML:** Thêm `PreviewTextInput` và `KeyUp` vào `ComboBox`.

   ```
   xml
   <ComboBox x:Name="SearchComboBox"
             Height="30"
             IsEditable="True"
             PreviewTextInput="SearchComboBox_PreviewTextInput"
             KeyUp="SearchComboBox_KeyUp"
             IsDropDownOpen="{Binding IsDropDownOpen, Mode=TwoWay}">
   </ComboBox>
   ```

2. **Xử lý Sự kiện PreviewTextInput:** Trong sự kiện này, bạn sẽ lọc danh sách dựa trên văn bản người dùng nhập vào.

   ```
   csharp
   private void SearchComboBox_PreviewTextInput(object sender, TextCompositionEventArgs e)
   {
   ```

```
    UpdateSearchSuggestions();
}

private void UpdateSearchSuggestions()
{
    var searchText = SearchComboBox.Text.ToLower();
    var filteredItems = _lib.Where(book => book.BookName.ToLower().Contains(searchText)).ToList();

    SearchComboBox.ItemsSource = filteredItems;
    SearchComboBox.IsDropDownOpen = filteredItems.Any();
}
```

3. **Xử lý Sự kiện KeyUp:** Điều này cần thiết để cập nhật danh sách gợi ý khi người dùng xóa văn bản.

```
csharp
private void SearchComboBox_KeyUp(object sender, KeyEventArgs e)
{
    if (e.Key == Key.Back || e.Key == Key.Delete)
    {
        UpdateSearchSuggestions();
    }
}
```

4. **Lựa Chọn Từ Danh sách Gợi ý:** Sử dụng `SelectionChanged` để xử lý lựa chọn từ danh sách gợi ý.

```
xml
<ComboBox x:Name="SearchComboBox"
          ...
          SelectionChanged="SearchComboBox_SelectionChanged">
</ComboBox>
```

Và trong phương thức `SearchComboBox_SelectionChanged`:

```
csharp
private void SearchComboBox_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    // Xử lý lựa chọn của người dùng từ danh sách gợi ý
}
```

Lưu ý rằng việc triển khai này đòi hỏi bạn phải cẩn thận khi xử lý các trường hợp khác nhau như nhập văn bản, xóa văn bản và chọn từ danh sách gợi ý. Bạn cũng cần đảm bảo rằng `IsDropDownOpen` của `ComboBox` được quản lý một cách chính xác để danh sách gợi ý hiển thị và ẩn theo hành động của người dùng.

You:

Sau khi thực hiện đoạn code trên, khi tôi nhập dữ liệu vào để tìm kiếm sách thì bị lỗi như vậy

---

ChatGPT:

Hình ảnh bạn chia sẻ cho thấy danh sách gợi ý hiển thị tên đầy đủ của đối tượng `Product` thay vì chỉ hiển thị thuộc tính cụ thể như `BookName`. Điều này xảy ra vì `ComboBox` mặc định sẽ gọi phương thức `ToString()` trên mỗi đối tượng trong `ItemsSource` để xác định nội dung hiển thị. Nếu bạn không ghi đè phương thức `ToString()` trong class `Product`, bạn sẽ nhận được tên đủ của class đối tượng đó.

Để hiển thị tên sách thay vì tên class, bạn cần thiết lập thuộc tính `DisplayMemberPath` của `ComboBox` để chỉ định thuộc tính nào của đối tượng `Product` sẽ được hiển thị.

Sửa đổi `ComboBox` trong file .xaml của bạn như sau:

```
xml
<ComboBox x:Name="SearchComboBox"
          Height="30"
          IsEditable="True"
          PreviewTextInput="SearchComboBox_PreviewTextInput"
          KeyUp="SearchComboBox_KeyUp"
          DisplayMemberPath="BookName"  <!-- Chỉ định thuộc tính để hiển thị -->
          IsDropDownOpen="{Binding IsDropDownOpen, Mode=TwoWay}">
</ComboBox>
```

Thuộc tính `DisplayMemberPath` được đặt thành "BookName", vì vậy bây giờ `ComboBox` sẽ hiển thị giá trị của thuộc tính `BookName` từ mỗi `Product` trong danh sách gợi ý thay vì tên class `Product`. Đảm bảo rằng `BookName` là tên chính xác của thuộc tính trong class `Product` mà bạn muốn hiển thị.

You:

Tôi không muốn phải ấn nút giống như combobox nó mở xổ xuống danh sách. Khi nhập vào kí tự tới đâu nó sẽ tự động xổ xuống tới đó

ChatGPT:

Trong trường hợp này, bạn có thể sử dụng một `TextBox` để nhập văn bản và một `Popup` để hiển thị danh sách gợi ý. `Popup` sẽ chứa một `ListBox` để hiển thị các gợi ý, và nó sẽ được mở ra tự động khi người dùng nhập văn bản vào `TextBox`.

Dưới đây là cách bạn có thể thiết lập `TextBox` và `Popup` trong XAML:

```
xml
<TextBox x:Name="SearchTextBox"
         Width="200"
         KeyUp="SearchTextBox_KeyUp" />

<Popup x:Name="SuggestionsPopup"
       PlacementTarget="{Binding ElementName=SearchTextBox}"
       Placement="Bottom"
       StaysOpen="False"
       IsOpen="{Binding Path=IsKeyboardFocusWithin,
                        RelativeSource={RelativeSource Mode=FindAncestor, AncestorType={x:Type TextBox}},
                        Mode=OneWay}">
    <ListBox x:Name="SuggestionsListBox"
             Width="{Binding ActualWidth, ElementName=SearchTextBox}"
             MaxHeight="200"
             IsSynchronizedWithCurrentItem="True"
             DisplayMemberPath="BookName" />
</Popup>
```

Trong code-behind, bạn sẽ cập nhật danh sách gợi ý trong sự kiện `KeyUp` của `TextBox`:

```
csharp
private void SearchTextBox_KeyUp(object sender, KeyEventArgs e)
{
    string searchText = SearchTextBox.Text.ToLower();

    if (string.IsNullOrWhiteSpace(searchText))
    {
        SuggestionsPopup.IsOpen = false;
        return;
    }

    var filteredItems = _lib.Where(book => book.BookName.ToLower().Contains(searchText)).ToList();

    if (filteredItems.Any())
    {
        SuggestionsListBox.ItemsSource = filteredItems;
        SuggestionsPopup.IsOpen = true;
    }
    else
    {
        SuggestionsPopup.IsOpen = false;
    }
}
```

Để xử lý việc người dùng chọn một gợi ý từ `ListBox`, bạn có thể thêm sự kiện `SelectionChanged` cho `ListBox`:

```
xml
<ListBox x:Name="SuggestionsListBox"
         ...
         SelectionChanged="SuggestionsListBox_SelectionChanged" />
```

Và xử lý
